

Gestión de la memoria

La memoria principal o *Random Access Memory (RAM)* consiste en un arreglo de bytes. Cada byte se *identifica* por su *posición* o *dirección* en el arreglo. Esta memoria es volátil, es decir que los datos almacenados se pierden cuando se corta la energía. La cpu interactúa directamente con la RAM en cada ciclo de ejecución de cada instrucción. En las arquitecturas del tipo Von-Neuman, las instrucciones de programa y los datos se almacenan en la misma memoria.

En un sistema multitarea la memoria se debe *particionar lógicamente* para separar las instrucciones y datos de cada programa en ejecución. Esto requiere que mínimamente un SO deba gestionar cómo se asigna memoria al kernel y a los procesos y además utilizar mecanismos de protección para que cada proceso se ejecute *confinado*, es decir que sólo pueda acceder a sus áreas de memoria con el fin de evitar que un proceso pueda invadir las áreas de los otros procesos o del kernel.

La cpu ejecuta repetidamente el *ciclo de instrucción* que comúnmente tiene tres pasos:

1. *Fetch (lectura) de la instrucción*: Se carga en un registro interno la instrucción con la dirección de memoria del *program counter (pc)*.
2. *Decodificación*: Se identifica el tipo de instrucción y sus operandos.
3. *Ejecución* de la instrucción. Esto puede requerir *leer* o *escribir* en la memoria.

Jerarquía de memorias

La cpu comúnmente usa registros (memoria de alta velocidad) para la ejecución de las instrucciones. Es común que una operación de lectura o escritura desde/hacia la memoria tome varios ciclos de reloj, haciendo que la cpu deba *frenarse (stall)*. Para atenuar estas diferencias de velocidades se usan memorias *caché*: Memorias de menor capacidad pero de más alta velocidad de acceso que la RAM convencional.

Una memoria caché está diseñada para el acceso a los datos *por contenido* en lugar de *direcciones*. Cuando la cpu hace referencia a una dirección de memoria, si el contenido de esa dirección está en la caché, se recupera desde allí. Si no se encuentra en la caché, se accede a la RAM y se almacena en la caché para accesos futuros.

Finalmente, para almacenar programas y datos en forma permanente se usan dispositivos de almacenamiento como discos magnéticos, de estado sólido u otra tecnología. Esta arquitectura define una *jerarquía* de memorias como se muestra en la siguiente figura ordenadas por velocidad de acceso de izquierda a derecha.

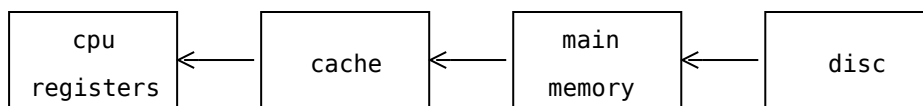


Figura 1: Jerarquías de memoria.

Asignación de memoria

El kernel deberá asignar y liberar bloques de memoria a los procesos y para las estructuras de datos del kernel mismo.

Una vez que el kernel se cargó en memoria durante el proceso de carga (*boot*), deberá reconocer el área de memoria libre disponible para asignar. Esta área de memoria se conoce como *heap*.

Comúnmente se usan algunas de las dos principales estrategias de gestión del heap.

1. Asignación de bloques de tamaño variable.
2. Asignación de bloques de tamaño fijo.

En el primer caso el heap se particiona lógicamente y se asignan bloques de tamaño variable. En el segundo se particiona y asignan bloques de tamaño fijo.

Heap con bloques de tamaño variable

La primera estrategia comúnmente se implementa con una lista de bloques libres de la siguiente forma.

- Inicialmente existe un único bloque libre del tamaño del heap.
- Ante una solicitud de un bloque de tamaño n , se busca el bloque libre b con tamaño $b.size \geq n$. Si $b.size > n$, se particiona y se asigna el bloque de tamaño n quedando un bloque libre b' con $b'.size = b.size - n$.

Si no existe un bloque tal, se retorna un error.

- Ante una liberación de un bloque, se inserta en la lista de bloques libres.

Obviamente, el algoritmo de búsqueda de un bloque que satisfaga el requerimiento afectará el rendimiento del sistema. Algunas estrategias para reducir la complejidad (tiempo) de la búsqueda pueden ser:

1. *First fit*: Se asigna el primer bloque con tamaño $\geq n$.

2. *Best fit*: Se asigna el bloque que *mejor ajusta*, es decir un bloque b con $b.size = m$, $m \geq n$ y $\forall b' \in freelist \mid b'.size \geq b.size$.

3. *Worst fit*: Se asigna el bloque libre más grande.

A medida que se hacen los requerimientos y liberaciones de bloques, el heap se puede ir *fragmentando* o particionando en un gran número de bloques libres pequeños. Esto impediría que se puedan satisfacer requerimientos de bloques grandes. Este problema se conoce como *fragmentación externa*.

Una estrategia que puede aliviar la fragmentación es que ante una liberación de un bloque, si existen bloques libres adyacentes, se *fusionen* o *compacten* en un único bloque libre de mayor tamaño. Esta estrategia se conoce como *semi-compactación*.

Heap con bloques de tamaño fijo

En este caso el heap se particiona en m bloques de un tamaño fijo s .

Ante un requerimiento de un bloque de tamaño n :

- Si $n \leq s$ se asigna cualquier bloque libre.
- Sino, se deben asignar el mínimo de k bloques *contiguos* tal que $k \times s \geq n$.

Comúnmente las listas (conjuntos) de bloques libres y usados se representan como un *bit vector*, lo que es una gran ventaja en términos de uso de memoria.

En éste esquema, el tamaño s de los bloques es crítico. Un valor grande podría aumentar la *fragmentación interna*, es decir el remanente de los bloques asignados que no se usarán. Un valor muy pequeño podría requerir encontrar varios bloques libres contiguos. Si no se pueden encontrar, no se puede satisfacer el requerimiento a pesar que puede haber muchos bloques libres en el heap.

El sistema compañero (buddy system)

Para poder lograr los beneficios de las dos estrategias mencionadas, es posible diseñar alguna solución híbrida combinándolas.

El *buddy system* se basa en particionar el heap en bloques de tamaño desde 2^l hasta 2^h . Por ejemplo, con $l = 16$ y $h = 20$ los bloques pueden ser 64K, 128K, 256K y 1024K bytes. El algoritmo funciona de la siguiente manera:

1. Inicialmente, el heap se particiona en bloques de tamaño 2^h (1024KB).
2. Ante un requerimiento de tamaño n

1. Seleccionar un b con mejor ajuste.
2. Si $b.size \geq 2n$ dividir b en dos bloques b_0 y b_1 de tamaño $b.size/2$.

Sea $b = b_0$, repetir este paso hasta que Sea $b.size = 2^l$.
3. Asignar el bloque b .

En una liberación de un bloque se aplica el algoritmo a la inversa. Si quedan bloques contiguos (compañeros) de igual tamaño menores a 2^h , se unen en un bloque mas grande.

Protección

Es necesario que un proceso sólo pueda acceder a su área de memoria. Suponiendo que se asigna la memoria a un proceso en forma contigua, haría falta una maquinaria que permita la cpu *detectar* accesos fuera de rango. Este mecanismo podría basarse en un registro de control que contenga la ***dirección base*** y la ***dirección límite***. Además podría tener *flags* de *permisos de acceso*, como lectura, escritura, ejecución y otros.

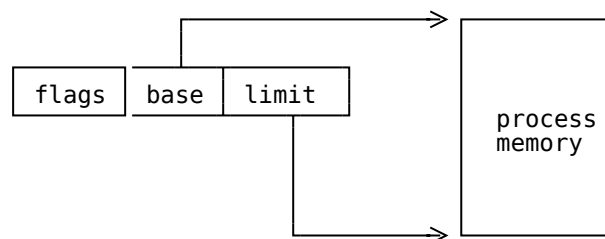


Figura 2: Registros *base* y *límite*.

Cuando el SO realiza un *context switch* a un proceso asigna las direcciones correspondientes a estos registros. La cpu en cada acceso a memoria verifica que la dirección se encuentre en el área especificada por los registros: $base \leq address \leq limit$.

En caso que en una ejecución de una instrucción refiera a una dirección fuera de los límites, o que la operación de lectura o escritura no cumpla con los permisos la cpu genera una *memory fault exception*.

Comúnmente, el SO seguramente atraparé la excepción mediante algún *exception handler* y si no es recuperable, eliminará el proceso. Este error comúnmente se observa como un *segmentation fault* en sistemas tipo UNIX.

Este mecanismo se puede extender al uso de varios de estos registros, para que las áreas o *segmentos* (*texto*, *datos*, *stack* y *heap*) de un proceso no necesariamente se deban almacenar en forma contigua (no siempre se va a encontrar un bloque libre tan grande). En este caso el mecanismo se conoce como *segmentación*. A modo de ejemplo, la arquitectura Intel IA32 (x86) soporta los siguientes registros de segmentos (base+límit): *code segment (CS)*, *data segment (DS)*, *stack segment (SS)*, *extra segment (ES)* y otros (*GS*, *FS*), como se muestra en la siguiente figura.

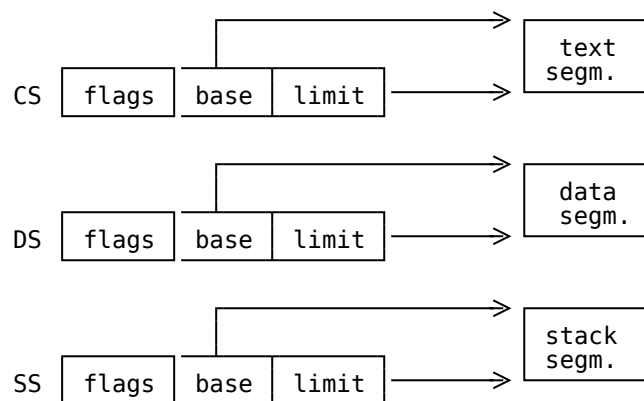


Figura 3: Registros de segmentos.

Código independiente de la posición (pic)

Con esta maquinaria, es posible lograr que el código de un programa pueda ejecutarse independientemente de la dirección base de carga. Esto puede lograrse con *código independiente de su posición* y básicamente consiste en que las direcciones de memoria (operandos de las instrucciones) representan *direcciones relativas* (con respecto a una dirección base) en lugar de direcciones absolutas.

Los registros de segmentos contienen las direcciones base.

Así la cpu debe *transformar* una dirección relativa en una *dirección física* sumándole el valor de la dirección base del segmento.

A modo de ejemplo, en x86, una dirección de memoria en una instrucción se denota como `segment:offset`. Una instrucción de salto del tipo `jmp address`, es equivalente a `jmp cs:address`, una instrucción de lectura/escritura desde/a memoria (ej: `mov %eax, address`) equivale a `mov %eax, ds:address`. Las instrucciones sobre la pila (`push/pop`) refieren por omisión al segmento `ss`.

Paginado

Como se verá en el siguiente capítulo, la implementación de algunas políticas de *memoria virtual* se simplifican si la memoria se particiona en bloques pequeños (*páginas*) del mismo tamaño, típicamente de 4KB como se muestra en la siguiente figura.

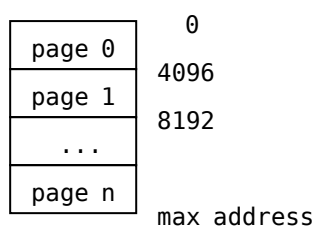


Figura 4: *Particionado lógico de la memoria en páginas de 4KB.*

Con paginado es posible definir ***espacios de memoria*** para cada proceso (y el kernel). Un espacio de memoria está representado por el conjunto de las páginas a las que puede acceder. Estos conjuntos generalmente se representan en la memoria como arreglos y se conocen como *tablas de páginas*.

Las páginas asignadas a un proceso no necesitan estar *físicamente contiguas*, aunque se haya definido un espacio de direcciones lógicas contiguas.

Esto requiere de un mecanismo de traducciones de *direcciones lógicas o virtuales a direcciones físicas o reales*. Esto puede verse como una función $map : va \rightarrow pa$, donde *va* es una *dirección lógica (virtual address)* y *pa* es una *dirección física (physical address)*.

Comúnmente el hardware con soporte de paginado implementa esta función de mapping con una *tabla de páginas* que representa la función de traducción de direcciones virtuales a físicas por extensión (o por casos).

Por cada proceso y para el kernel, el SO construye sus propios espacios de memoria asociando una *tabla de páginas* a cada uno. Cada entrada en una *page table* se denomina *page table entrie (pte)* y básicamente es un par de la forma $(ppn, flags)$. El número de página *ppn (physical page number)* refiere al número de página física y los *flags* son *permisos de acceso y de control*.

Direcciones virtuales

En paginado, una dirección en una instrucción de programa es en realidad una *dirección lógica* y generalmente se interpreta como un par $(index, offset)$.

El *index* refiere a una entrada en la tabla de páginas. La dirección física se obtiene computando

$$pa = pagetable[index].ppn * PS + offset$$

como se muestra en la siguiente figura, donde *PS* es el *page size*.

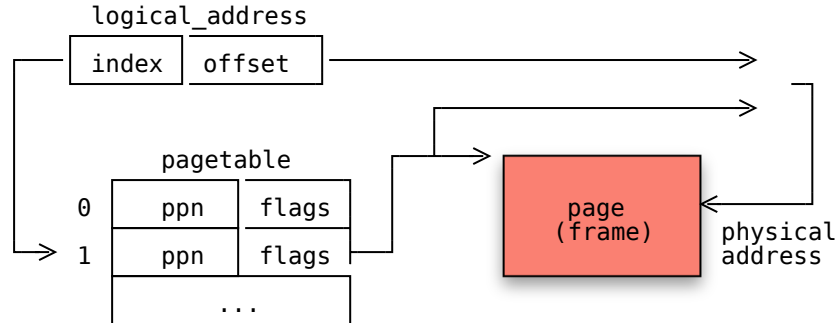


Figura 5: *Tablas de página.*

El *offset* contiene tantos bits como sea necesario dependiendo del tamaño de página. Por ejemplo si el tamaño de página es 4096, *offset* debería tener 12 bits ($2^{12} = 4096$).

La cpu genera una excepción del tipo *page fault* en el caso que una dirección lógica indexa en una entrada sin permisos de acceso o inválida.

Con paginado es posible que cada proceso tenga por ejemplo un espacio de direcciones de la forma $[0, size]$ donde *size* es el tamaño de la memoria requerida por el proceso. Así, todos los programas se compilan con opciones de memoria *por default* y pueden coexistir en memoria referenciando las mismas direcciones lógicas ya que en ejecución se traducirán en cada caso a direcciones físicas diferentes.

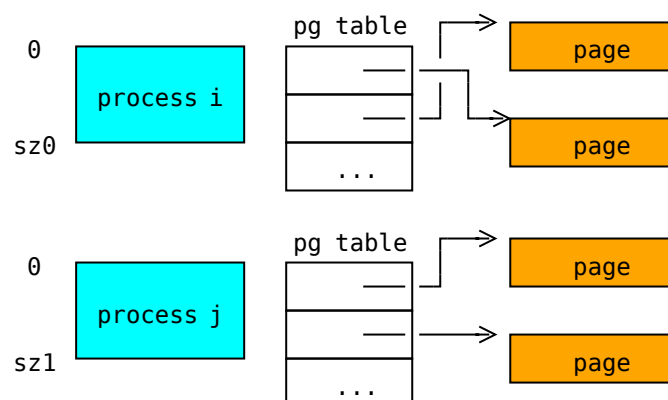


Figura 6: *Procesos con espacios de direcciones lógicas no disjuntas.*

Se debe notar que cuando un proceso usa paginado un *bloque de memoria con páginas con direcciones virtuales contiguas* no necesariamente se corresponden a bloques de direcciones físicas contiguas. Esto permite que un *heap allocator* puede asignar páginas a un proceso con cualquier dirección física, la cual será *mapeada* a una *dirección lógica* del proceso, como se muestra en la siguiente figura.

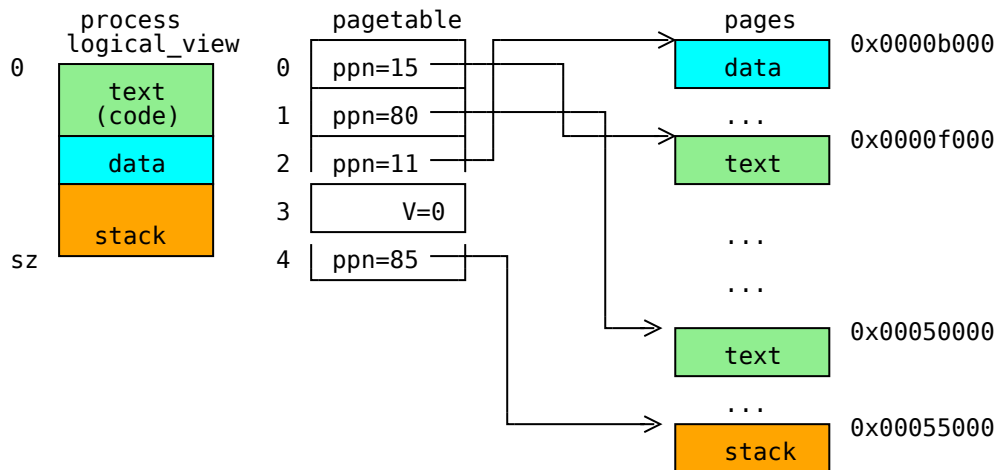


Figura 7: Mapping de direcciones lógicas a físicas.

En la figura anterior se puede apreciar que el *espacio contiguo de direcciones lógicas* se mapea un *conjunto de espacios de direcciones físicas*.

En este ejemplo, el proceso tiene dos páginas (8KB) para el área de código, 1 de datos y 1 para el stack. Se deja un *hueco* en el espacio de direcciones virtuales para detectar *stack overflow*.

Se debe notar que las páginas de código y datos son lógicamente contiguas pero físicamente están dispersas en la memoria.

Para los siguientes ejemplos, asumiremos que una dirección de memoria es de 32 bits y que se usan los 20 bits más significativos como *index* y los 12 menos significativos como *offset* en páginas de 4KB.

En la figura anterior, el espacio de direcciones lógicas del proceso es el intervalo $[0, sz = 0x5000]$ (20KB). Cuando una instrucción del programa haga referencia a la dirección lógica 0x2004 (área de datos) la cpu la interpreta con *index=2* y *offset=0x004*. La *pte* en la entrada 2 contiene el *physical page number=11*. La dirección física final será $11 * 4096 + 004 = 0xb004$.

Las tablas de páginas deberían tener suficientes entradas como para *barrer* toda la memoria direccionable por el sistema. Comúnmente esto es necesario en un kernel, ya que debe poder acceder a toda la memoria.

Con espacios direcciones grandes, por ejemplo 4GB en una arquitectura de 32 bits, el número de entradas en la tabla de páginas puede ser considerable.

En estos casos comúnmente el hardware permite definir tablas de páginas que refieran a páginas de mayor tamaño, como 4MB o más. Esto permite reducir el número de entradas en una tabla de páginas.

Aún así comúnmente es deseable que las páginas sean pequeñas para minimizar la *fragmentación interna*. Además un proceso puede dejar muchos *huecos* en el espacio de direcciones lógicas, lo que una tabla *plana* Esto requiere que la representación de una tabla de páginas no

Esto hace que las arquitecturas comúnmente representen la tabla como árboles de n niveles como en la siguiente figura.

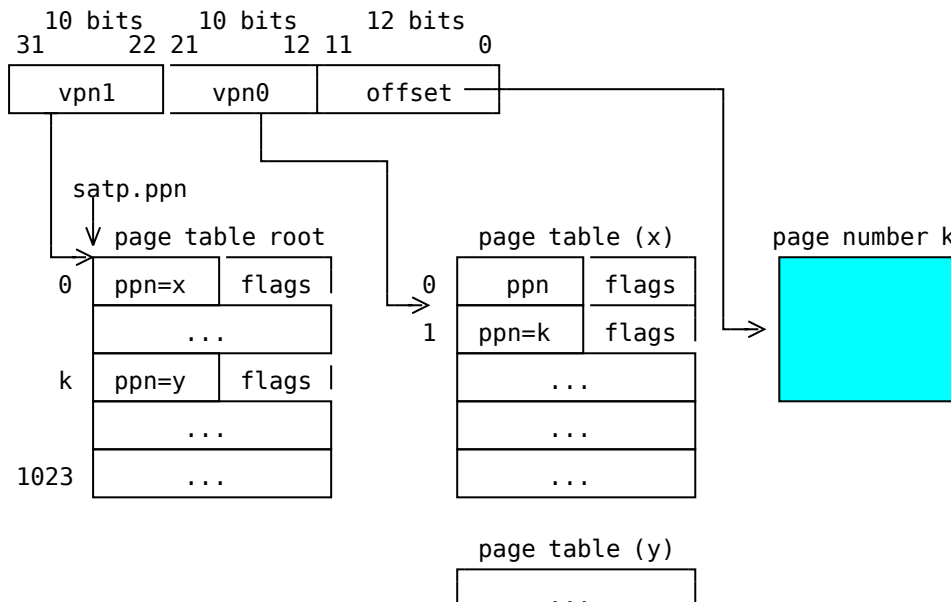


Figura 8: *Tablas de páginas en risc-v RV-32.*

Cada nodo del árbol tiene el tamaño de una página.

La figura de arriba muestra un ejemplo de una tabla de páginas en riscv (RV-32), una arquitectura con un espacio de direcciones de 32 bits (4GB). Una tabla de páginas tiene la forma de un árbol de dos niveles. El registro **satp** contiene el número de página física de la raíz del árbol.

Una dirección lógica se divide en 3 partes: **vpn1** (10 bits más significativos) es un índice o *virtual page number (vpn)* en el nodo raíz del árbol cuya *pte* contiene el *ppn* correspondiente a un nodo hoja, **vpn0** es el índice en el nodo hoja (cuya *pte* contiene el *ppn* de la página destino). Los 12 bits menos significativos son el *offset* dentro de la página destino.

La figura 8 muestra una dirección lógica de la forma **0x0000100f** que se interpreta como

vpn1	vpn0	offset
0000000000	0000000001	000000001111

Se debe notar que un nodo de la tabla de páginas incluye 1024 entradas, cada una ocupando 32 bits. Así son necesarios 10 bits para indexar una entrada en un nodo ($2^{10} = 1024$).

Cada entrada de una tabla de páginas (*page table entry - pte*) en RV-32 tiene la forma

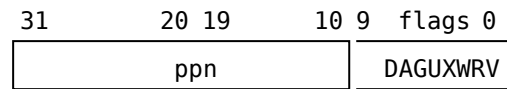


Figura 8: Formato de una *page table entry (pte)*.

En ésta configuración la función de traducción de una dirección virtual *va* a una dirección física *pa* sería:

$$pa = ((satp.ppn[va.vpn1].ppn * PS)[va.vpn0].ppn * PS) + va.offset$$

donde *satp.ppn* denota el valor del campo de la *physical page number* del registro CSR *supervisor address translation an protection (satp)*, *PS* es el *page size*.

La expresión $(satp.ppn[va.vpn1].pn * PS)$ referencia la tabla de páginas hoja (*leaf*) y la expresión $leaf[va.vpn0].pn * PS$ referencia la página física destino.

RV-39

En la arquitectura RV-39 usada en el taller de xv6, el espacio de direcciones físicas es de 39 bits y una tabla de páginas se implementa como un árbol de tres niveles. Una dirección virtual se descompone de la forma $(ppn2, ppn1, ppn0, offset)$, con *ppn_i* de 9 bits y *offset* de 12 bits. Cada nodo (página de 4KB) contiene 512 *ptes* de 64 bits cada una. La dirección física se traduce de manera similar que en RV-32 pero *caminando* el árbol por el nivel intermedio.

El kernel deberá cambiar el `sapt` en cada *context switch* para que reference la tabla de páginas del nuevo proceso seleccionado.

Los *flags* en una *pte* comúnmente tienen los siguientes bits:

- **V:** *Valid* bit. Indica si la entrada es válida.
- **D:** *Dirty* bit. Encendido por el hardware cuando la página correspondiente fue modificada (escrita) por una instrucción.
- **A:** *Accesed* bit. Encendido por el hardware cuando una página fue accedida para lectura o escritura.
- **U:** *User mode* bit. Permiso de acceso en *user mode*.
- Los bits **R, W, X** determinan permisos de lectura, escritura o ejecución de la página.

En el proceso de traducción de una dirección lógica a una física, la cpu verifica los correspondientes flags. En caso de una violación, por ejemplo, si $V=0$ genera un *page fault*. El kernel podrá atrapar estas excepciones y actuar en consecuencia.

En el capítulo [memoria virtual](#) se describen técnicas de gestión de memoria virtual que hacen uso de estos mecanismos.

Lookaside translation buffers

Un sistema con paginado traduce constantemente direcciones lógicas a físicas usando las tablas de páginas. Estas tablas están almacenadas en RAM. En este esquema cada acceso a memoria se debería multiplicar por 4: acceso a la *pte raíz*, *pte intermedia* y *pte de la hoja* para finalmente acceder a la dirección física final.

Para evitar esta sobrecarga las cpu modernas usan una memoria caché especial para almacenar las *ptes* más frecuentemente referenciadas. Esta caché especial se conoce como *lookaside translation buffers* o *TLBs*. La TLB reside comúnmente entre la CPU y la caché.

Una TLB es comúnmente una *memoria direccionable por contenido*, es decir que busca una dirección lógica y si ésta existe (*hit*), el resultado es una dirección física, en otro caso ocurre un *miss*.

Generalmente implementan una *función hash* donde la clave de búsqueda son una parte de los *bits menos significativos de la virtual page number* y está asociada a un *tag* el cual es el conjunto de bits mas significativo de la *vpn*. El par (*tag, index*) identifica una *virtual page number*. Esta organización se conoce como *direct mapping* y se muestra en el siguiente diagrama.

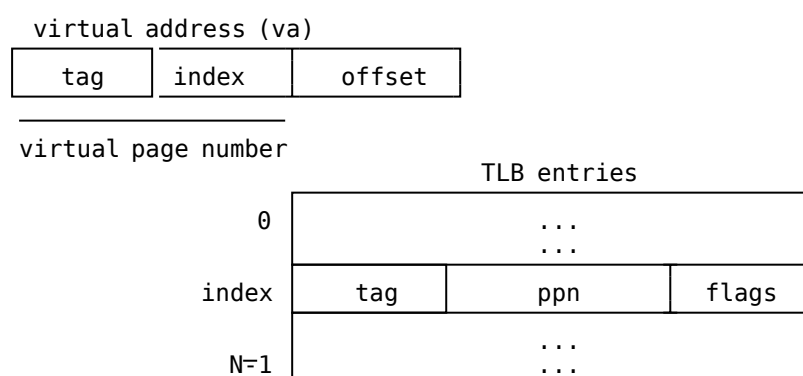


Figura 9: TLB con direct mapping.

Cada *TLB entry* contiene el *tag*, el *physical page number (ppn)* y flags, entre los que se encuentra el bit *valid (V)* que indica si la entrada es válida o no.

En el caso que $TLB[index].tag == va.tag$ y $TLB[index].flags \& BIT_V \neq 0$ ocurre un *hit* y $TLB[index].ppn$ contiene la *physical page number*.

En otro caso ocurre un *miss* y la cpu debe acceder a la RAM (o caché) a la tabla de páginas y obtener la *ppn* correspondiente. Luego, se setea $TLB[index]$ apropiadamente para futuras referencias.

Algunas arquitecturas, como MIPS, permiten que el kernel decida (por software) qué hacer ante un *miss*.

[< Anterior](#)

Concurrencia e IPC

[Próximo >](#)

Memoria virtual